

AUDITORÍA DE RECURSOS COMPARTIDOS CRÍTICOS

PROYECTO EDARSAHUB

Fecha: 08/04/2026 12:32

1. RECURSO: /api/servers

A. Módulos que lo usan:

Módulo	Archivo	Líneas de Uso	Método
Comercial	Comercial.js	2365	axios directo
Compras	Compras.js	2760	axios directo
Reportes	Reportes.js	648	api.js wrapper
Dashboard	Dashboard.js	103	api.js wrapper
Explorador BD	ExploradorBD.js	828	axios directo
Catálogo SQL	CatalogoConsultas.js	98	axios directo
Autorización Compras	AutorizacionCompras.js	87	axios directo
Servidores	Servidores.js	91, 200, 207, 237, 291, 337	api.js wrapper
Usuarios	Usuarios.js	92	api.js wrapper
Mis Tareas	MisTareas.js	167	fetch directo
Proveedores	Proveedores.js	57	fetch directo

B. Consumo:

- **Directo (axios/fetch):** Comercial, Compras, ExploradorBD, CatálogoSQL, AutorizaciónCompras, MisTareas, Proveedores
- **Via api.js wrapper:** Reportes, Dashboard, Servidores, Usuarios
- **Total módulos:** 11 de 17 módulos (65%)

C. Lógica Duplicada:

El siguiente patrón se repite en 7 archivos:

```
const response = await axios.get(`${API_URL}/api/servers`, { headers: { Authorization: `Bearer ${token}` } });
const serversOperativos = response.data.filter(s => s.visible_en_operaciones !== false);
```

D. Nivel de Riesgo: **CRÍTICO**

E. Riesgo de Cambio:

- **Cambiar estructura de respuesta → rompe 11 módulos**
- **Cambiar nombre del campo 'visible_en_operaciones' → rompe 7 módulos**
- **Agregar campos nuevos → sin riesgo**

F. Recomendación:

- ✓ CONGELAR endpoint completamente
- ✓ Crear servicio compartido fetchServersOperativos() para eliminar duplicación
- ✓ NO mover a Context Global aún - cada módulo maneja su estado

2. RECURSO: /api/servers/{id}/sucursales

A. Módulos que lo usan:

Módulo	Archivo	Línea	Endpoint Específico
Comercial	Comercial.js	2388	/api/servers/{id}/sucursales
Comercial	Comercial.js	1677	/api/comercial/sucursales/{id} (variante)
Compras	Compras.js	2790	/api/servers/{id}/sucursales
Reportes	Reportes.js	670	/api/servers/{id}/sucursales
Autorización Compras	AutorizacionCompras.js	218	/api/servers/{id}/sucursales
Usuarios	Usuarios.js	406	/api/servers/{id}/sucursales
Finanzas	Finanzas.js	110	/api/rrhh/catalogos/sucursales (otro)
Nóminas	Nominas.js	117	/api/rrhh/catalogos/sucursales (otro)
Recursos Humanos	RecursosHumanos.js	197	/api/rrhh/catalogos/sucursales (otro)

B. Problema Detectado:

- Existen 3 endpoints diferentes para obtener sucursales:
 1. /api/servers/{id}/sucursales (SQL Server externo)
 2. /api/comercial/sucursales/{id} (variante en Comercial)
 3. /api/rrhh/catalogos/sucursales (MongoDB local)

C. Nivel de Riesgo: MEDIO-ALTO

D. Riesgo de Cambio:

- **Cambiar formato de respuesta {id, nombre, codigo} → rompe 5 módulos**
- Los módulos de RRHH usan endpoint diferente → menor riesgo cruzado

E. Recomendación:

- ✓ CONGELAR estructura de respuesta {id, nombre, codigo}
- ✓ Mantener los 3 endpoints separados (ya están aislados por dominio)

3. RECURSO: visible_en_operaciones (Filtro)

A. Módulos que lo usan:

Módulo	Archivo	Línea	Uso
Comercial	Comercial.js	2370	Filtro en fetch servers
Compras	Compras.js	2765	Filtro en fetch servers
Reportes	Reportes.js	653	Filtro en fetch servers
Dashboard	Dashboard.js	106	Filtro en fetch servers
Explorador BD	ExploradorBD.js	833	Filtro en fetch servers
Catálogo SQL	CatalogoConsultas.js	103	Filtro en fetch servers
Autorización Compras	AutorizacionCompras.js	92	Filtro en fetch servers
Servidores	Servidores.js	52, 236, 568-580	UI Toggle para editar

B. Lógica Duplicada:

El patrón `s.visible_en_operaciones !== false` se repite IDÉNTICAMENTE en 7 archivos

C. Backend:

- Definido en modelo Server (línea 111, 128)
- Usado en endpoint `/dashboard/servers-configured` (línea 5244)
- Usado en tablero-ejecutivo para filtrar (línea 10302)

D. Nivel de Riesgo: CRÍTICO

E. Riesgo de Cambio:

- **Renombrar el campo → rompe 8 archivos frontend + 4 backend**
- **Cambiar lógica de True/False → inconsistencias en filtrado**

F. Recomendación:

- ✓ CONGELAR nombre del campo
- ✓ Crear función compartida `filterServersOperativos(servers)` para eliminar duplicación

4. RECURSO: localStorage.token

A. Estadísticas de Uso:

Archivo	Veces usado	Método
Comercial.js	27	localStorage.getItem directo
Compras.js	44	localStorage.getItem directo
ExploradorBD.js	50+	localStorage.getItem directo
AutorizacionCompras.js	10	localStorage.getItem directo
CatalogoConsultas.js	7	localStorage.getItem directo
MisTareas.js	20+	fetchWithAuth (helper interno)
Reportes.js	5	api.js interceptor
Dashboard.js	3	api.js interceptor
Servidores.js	5	api.js interceptor
Usuarios.js	8	api.js interceptor

B. Problema Detectado:

- 65% de módulos usan localStorage.getItem('token') DIRECTAMENTE
- Solo 35% usa el wrapper api.js que maneja token automáticamente
- Total de llamadas directas: 150+ en todo el proyecto

C. Nivel de Riesgo: MEDIO

D. Riesgo de Cambio:

- **Cambiar nombre de 'token' en localStorage → rompe TODOS los módulos**
- **Cambiar estructura de autenticación → requiere tocar 150+ líneas**
- Mantener como está → funciona pero hay código duplicado masivo

E. Recomendación:

- ✓ CONGELAR key 'token' en localStorage
- ✓ Migrar gradualmente a api.js (no urgente, solo nuevos desarrollos)
- ✗ NO hacer migración masiva - alto riesgo de regresión

5. RECURSO: `execute_sql_query()` [Backend]

A. Estadísticas:

- Definición: Línea 503 de `server.py`
- Total de llamadas: **176 veces** en todo el backend
- Es el ÚNICO punto de contacto con SQL Server externos

B. Módulos/Endpoints que lo usan:

Categoría	Endpoints	# Llamadas
Compras	<code>/compras/*</code> , <code>/inventarios/*</code>	45+
Comercial	<code>/comercial/*</code>	30+
Dashboard	<code>/dashboard/*</code>	15+
Servidores	<code>/servers/*</code> /sucursales, almacenes, etc	25+
Reportes	<code>/reportes/*</code>	20+
RRHH/Nóminas	<code>/rrhh/*</code> , <code>/nomina/*</code>	15+
Otros	Varios endpoints auxiliares	26+

C. Nivel de Riesgo: **MÁXIMO CRÍTICO**

D. Riesgo de Cambio:

- **Cambiar firma de la función** → rompe 176 llamadas
- **Cambiar manejo de errores** → comportamiento impredecible en todo el sistema
- **Cambiar timeout default** → afecta rendimiento global

E. Recomendación:

- ✓ CONGELAR COMPLETAMENTE - NO TOCAR BAJO NINGUNA CIRCUNSTANCIA
- ✓ Si se necesita modificar, crear `execute_sql_query_v2()` nuevo

6. RECURSO: `get_current_user()` [Backend]

A. Estadísticas:

- Definición: Línea 315 de `server.py`
- Total de usos: **183 endpoints** usan `Depends(get_current_user)`
- Es la ÚNICA función de autenticación del sistema

B. Nivel de Riesgo: **MÁXIMO CRÍTICO**

C. Riesgo de Cambio:

- **Cambiar estructura del Dict retornado** → rompe lógica en 183 endpoints
- **Cambiar validación de token** → bloquea acceso a todo el sistema

D. Recomendación:

- ✓ CONGELAR COMPLETAMENTE

✓ Mantener estructura: {id, email, role, name, ...}

RESUMEN: MATRIZ DE DEPENDENCIAS CRÍTICAS

Recurso	Módulos	Llamadas	Riesgo	Acción
/api/servers	11	~20	CRÍTICO	CONGELAR
/api/servers/{id}/sucursales	9	~15	ALTO	CONGELAR
visible_en_operaciones	8	~15	CRÍTICO	CONGELAR
localStorage.token	17	150+	MEDIO	CONGELAR key
execute_sql_query()	Backend	176	MÁXIMO	NO TOCAR
get_current_user()	Backend	183	MÁXIMO	NO TOCAR

CONCLUSIÓN GENERAL:

El sistema tiene 6 recursos compartidos críticos que deben considerarse INMUTABLES.

Cualquier cambio en estos recursos requiere:

1. Análisis de impacto en todos los módulos dependientes
2. Pruebas de regresión completas
3. Migración gradual (si es absolutamente necesario)
4. Versionado de endpoints (v2) en lugar de modificar existentes

--- Fin del Documento ---